

Fatio, Rationally Reconstructed: A Faithful Deep and Shallow Embedding of a Multi-Agent Argumentation Protocol

Christoph Benz Müller Luca Pasetto

August 2, 2026

Abstract

We present a rational reconstruction of the LogiKEy implementation [7] of the Fatio dialogue protocol for multi-agent dialectical argumentation [6], rebuilt following the deep and shallow embedding methodology of [2] and its formal development [1]. Both object logics involved — the propositional content logic and the multi-agent belief/desire logic over it, enriched with Done-, entailment- and existential-justification atoms — are developed as deep embeddings (datatype syntax with primitive-recursive semantics over models with explicit world domains) and as shallow ones, in a maximal (model-parametric) and a minimal (LogiKEy-style) variant [3], all connected by machine-checked faithfulness theorems.

Because the embeddings are faithful, the protocol layer can be defined over the deep syntax while semantic work is done on either shallow backend. This lifts three simplifications of the earlier formalisation, each documented and each compared with its predecessor by a theorem: the pre-condition of question is sign-sensitive, so that a challenger can be questioned; the state update filters formulas that clash with what a locution asserts, which matters because an inconsistent state trivialises every pre-condition; and question and challenge carry the post-conditions the axiomatic semantics prescribes. Fatio’s obligation to respond becomes checkable, and both dialogues are shown to leave no request open. The entry also records what it does not establish: the converse of the quantifier-scope implication is refuted at the level of the consequence relation by a verified two-world model (found with *nitpick* [5]), the world knowledge of both dialogues is shown satisfiable and non-trivial so that no check succeeds vacuously, ill-formed dialogues are rejected, and an optional bridge relating entailment atoms to the content logic makes precise why argumentative support must remain defeasible.

Contents

1 Preliminaries

3

2	Deep embedding: content logic, Fatio language, BD logic	3
2.1	Content logic (deeply embedded, propositional)	3
2.2	The Fatio language	4
2.3	BD logic (deeply embedded)	4
2.4	Kripke semantics over models $\langle W, B, D, V, U, E \rangle$	4
3	Shallow embedding (maximal) of the content and BD logics	5
4	Shallow embedding (minimal) of the BD logic	6
5	Faithfulness	7
5.1	Faithfulness of the content layer	7
5.2	The content layer: faithfulness of its injection into the BD logic	8
5.3	Discrimination vs. identification: the Done-problem, resolved	8
5.4	Working on the shallow side: the transfer principle	8
5.5	The quantifier-scope question, settled	9
5.6	Faithfulness for the minimal embedding, and the comparison	10
5.7	Modal principles proved shallowly, used deeply	11
5.8	A worked illustration of the transfer	12
6	The Fatio protocol over the deep syntax	13
6.1	Dialectical obligation stores	13
6.2	Pre-conditions (axiomatic semantics)	13
6.3	Post-conditions and state evolution	14
6.4	Outstanding requests (Fatio's obligation to respond)	15
6.5	Sanity theorems	15
7	Variants: the earlier formalisation, and an optional argumentative bridge	16
7.1	Variant: the original, sign-blind question pre-condition	16
7.2	Variant: the earlier, unfiltered state update	17
7.3	Optional bridge: entailment atoms with content	18
8	Tests: two dialogues, checked with high automation	19
8.1	The full Brigade Restaurant dialogue (from the literature)	19
8.2	A further example, constructed here: role reversal with a negative retraction	21
8.3	Every request in the dialogues is answered	22
8.4	Why the dialogue needs the generalised question pre-condition	23
8.5	The examples are not vacuous	23
8.6	Negative tests: the checker discriminates	24
8.7	A pre-condition that does not follow by membership	24
8.8	The same, established on the shallow side	24

1 Preliminaries

```
theory FatioFaithful-preliminaries
  imports Main
begin
```

```
typedecl w — type of possible worlds
```

```
typedecl  $\mathcal{S}$  — type of propositional constant symbols of the content logic
```

```
datatype Speaker = a | b | c — the dialogue participants
```

```
type-synonym  $\mathcal{W} = w \Rightarrow bool$  — world domains
```

```
type-synonym  $\mathcal{R} = Speaker \Rightarrow w \Rightarrow w \Rightarrow bool$  — agent-indexed accessibility relations
```

```
type-synonym  $\mathcal{V} = \mathcal{S} \Rightarrow w \Rightarrow bool$  — valuations of content atoms
```

— Bounded universal quantifier, as in [1]

```
abbreviation(input) BoundedAll:: $\mathcal{W} \Rightarrow \mathcal{W} \Rightarrow bool$  where BoundedAll  $W \varphi \equiv \forall x. W x \longrightarrow \varphi x$ 
```

```
syntax -BoundedAll:: pttrn $\Rightarrow \mathcal{W} \Rightarrow bool \Rightarrow bool$  (( $\exists \forall (-/:-)/-$ ) [0, 0, 10] 10)
```

```
translations  $\forall x. W. \varphi \equiv CONST \text{BoundedAll } W (\lambda x. \varphi)$ 
```

```
end
```

2 Deep embedding: content logic, Fatio language, BD logic

```
theory FatioFaithful-deep
  imports FatioFaithful-preliminaries
begin
```

2.1 Content logic (deeply embedded, propositional)

SCOPE NOTE. The content logic is embedded deeply with a semantics of its own (see below), and shallowly in the shallow theory; the two are related by *Faithful0a*/*Faithful0b* in the faithfulness theory. Its meaning is moreover tied to the BD logic by the injection $\{-\}^d$, which is truth-preserving (*MapC-faithful*). The deep/shallow/faithfulness triangle of this entry therefore covers BOTH object logics. What the content layer deliberately does not carry is a notion of argumentative support: the entailment atoms of the BD logic are interpreted freely, and the optional bridge in the variants theory shows what constraining them by content-logic entailment would cost.

```
datatype Formula = AtmC  $\mathcal{S} (-^c [1000] 999) | \text{NegC } Formula (\neg^c - [96] 96)$ 
  | ImpC Formula Formula (infixr  $\supset^c$  93)
```

```
definition AndC (infixr  $\wedge^c$  95) where  $\varphi \wedge^c \psi \equiv \neg^c(\varphi \supset^c \neg^c \psi)$ 
```

```
definition OrC (infixr  $\vee^c$  92) where  $\varphi \vee^c \psi \equiv \neg^c \varphi \supset^c \psi$ 
```

```
definition TopC ( $\top^c$ ) where  $\top^c \equiv (SOME\ x. True)^c \supset^c (SOME\ x. True)^c$ 
```

```
definition BotC ( $\perp^c$ ) where  $\perp^c \equiv \neg^c \top^c$ 
```

— Kripke-style (here: valuation-based) semantics of the content logic

primrec *RelTC* :: $\mathcal{V} \Rightarrow w \Rightarrow \text{Formula} \Rightarrow \text{bool}$ ($-, - \models^c -$) **where**
 $V, w \models^c x^c = V\ x\ w$
 $| V, w \models^c \neg^c \varphi = (\neg\ V, w \models^c \varphi)$
 $| V, w \models^c \varphi \supset^c \psi = (V, w \models^c \varphi \longrightarrow V, w \models^c \psi)$

definition *ValC* ($\models^c -$) **where** $\models^c \varphi \equiv \forall V\ w. V, w \models^c \varphi$

2.2 The Fatio language

datatype *Sign* = *Pls* ($\oplus$) | *Mns* ($\ominus$)

datatype *FatioL* =

Assert *Speaker* *Formula* (*assert*[-,-])
 $|$ *Question* *Speaker* *Speaker* *Formula* (*question*[-,-,-])
 $|$ *Challenge* *Speaker* *Speaker* *Formula* (*challenge*[-,-,-])
 $|$ *Justify* *Speaker* *Formula* *Sign* *Formula* (*justify*[-,-,-])
 $|$ *Retract* *Speaker* *Formula* *Sign* (*retract*[-,-,-])

2.3 BD logic (deeply embedded)

The object language of the axiomatic semantics of Fatio [6], as formalised in [7]: agent beliefs $\mathcal{B}^d$ and desires $\mathcal{D}^d$ over the content logic, with three further kinds of atoms: *Done*^d for uttered locutions, *EntD* for argumentative entailment, and *ExJD* $i\ \varphi$ whose single semantic clause carries the existential of the pre-conditions of question and challenge, $(\exists \Delta)\ \text{Done}[\text{justify}(i, \Delta \vdash^{sg} \varphi)]$, inside the object language. Following the sign-parametric justify locution of the LogiKEy sources [3], *ExJD* carries the sign of the requested justification.

datatype *BDF* =

AtmD $\mathcal{S}$ ($-^a\ [1000]\ 999$)
 $|$ *DoneD* *FatioL* (*Done*^d)
 $|$ *EntD* *Formula* *Sign* *Formula*
 $|$ *ExJD* *Speaker* *Sign* *Formula*
 $|$ *NegD* *BDF* ($\neg^d\ -\ [96]\ 96$)
 $|$ *ImpD* *BDF* *BDF* (**infixr** $\supset^d\ 93$)
 $|$ *BelD* *Speaker* *BDF* ($\mathcal{B}^d$)
 $|$ *DesD* *Speaker* *BDF* ($\mathcal{D}^d$)

— Homomorphic injection of content formulas into the BD language

primrec *MapC* :: $\text{Formula} \Rightarrow \text{BDF}$ ($\{-\}^d$) **where**
 $\{\varphi^c\}^d = \varphi^a \mid \{\neg^c \varphi\}^d = \neg^d \{\varphi\}^d \mid \{\varphi \supset^c \psi\}^d = \{\varphi\}^d \supset^d \{\psi\}^d$

2.4 Kripke semantics over models $\langle W, B, D, V, U, E \rangle$

type-synonym $\mathcal{U} = \text{FatioL} \Rightarrow w \Rightarrow \text{bool}$

— Done-valuations

type-synonym $\mathcal{E} = \text{Formula} \Rightarrow \text{Sign} \Rightarrow \text{Formula} \Rightarrow w \Rightarrow \text{bool}$ — entailment-valuations

primrec *RelTD* :: $\mathcal{W} \Rightarrow \mathcal{R} \Rightarrow \mathcal{R} \Rightarrow \mathcal{V} \Rightarrow \mathcal{U} \Rightarrow \mathcal{E} \Rightarrow \mathcal{W} \Rightarrow BDF \Rightarrow bool$ ($\langle -, -, -, -, - \rangle, - \models^d -$) **where**

- | $\langle W, B, D, V, U, E \rangle, w \models^d x^a = V x w$
- | $\langle W, B, D, V, U, E \rangle, w \models^d Done^d l = U l w$
- | $\langle W, B, D, V, U, E \rangle, w \models^d EntD \Phi sg \varphi = E \Phi sg \varphi w$
- | $\langle W, B, D, V, U, E \rangle, w \models^d ExJD i sg \varphi = (\exists \Delta. U (Justify i \Delta sg \varphi) w)$
- | $\langle W, B, D, V, U, E \rangle, w \models^d \neg^d \varphi = (\neg \langle W, B, D, V, U, E \rangle, w \models^d \varphi)$
- | $\langle W, B, D, V, U, E \rangle, w \models^d \varphi \supset^d \psi = (\langle W, B, D, V, U, E \rangle, w \models^d \varphi \longrightarrow \langle W, B, D, V, U, E \rangle, w \models^d \psi)$
- | $\langle W, B, D, V, U, E \rangle, w \models^d \mathcal{B}^d i \varphi = (\forall v: W. B i w v \longrightarrow \langle W, B, D, V, U, E \rangle, v \models^d \varphi)$
- | $\langle W, B, D, V, U, E \rangle, w \models^d \mathcal{D}^d i \varphi = (\forall v: W. D i w v \longrightarrow \langle W, B, D, V, U, E \rangle, v \models^d \varphi)$

definition *ValD* ($\models^d -$) **where** $\models^d \varphi \equiv \forall W B D V U E. \forall w: W. \langle W, B, D, V, U, E \rangle, w \models^d \varphi$

— Global consequence from a set of deep premises, over all models

definition *ConsD* :: $(BDF \Rightarrow bool) \Rightarrow BDF \Rightarrow bool$ (**infix** $\models 25$) **where**

$$\Gamma \models \varphi \equiv \forall W B D V U E. (\forall \gamma. \Gamma \gamma \longrightarrow (\forall w: W. \langle W, B, D, V, U, E \rangle, w \models^d \gamma)) \longrightarrow (\forall w: W. \langle W, B, D, V, U, E \rangle, w \models^d \varphi)$$

named-theorems *DefD*

declare *ValC-def*[*DefD*] *AndC-def*[*DefD, simp*] *OrC-def*[*DefD, simp*] *TopC-def*[*DefD, simp*] *BotC-def*[*DefD, simp*]

ValD-def[*DefD*] *ConsD-def*[*DefD*] *ValC-def*[*DefD*]

end

3 Shallow embedding (maximal) of the content and BD logics

theory *FatioFaithful-shallow*

imports *FatioFaithful-deep*

begin

— Shallow embedding of the content logic: truth sets parametric in the valuation

type-synonym $\sigma_c = \mathcal{V} \Rightarrow \mathcal{W} \Rightarrow bool$

definition *AtmCS* :: $\mathcal{S} \Rightarrow \sigma_c$ ($-^{cs}$ [1000] 999) **where** $x^{cs} \equiv \lambda V w. V x w$

definition *NegCS* :: $\sigma_c \Rightarrow \sigma_c$ ($\neg^{cs}$ - [96] 96) **where** $\neg^{cs} \varphi \equiv \lambda V w. \neg \varphi V w$

definition *ImpCS* :: $\sigma_c \Rightarrow \sigma_c \Rightarrow \sigma_c$ (**infixr** $\supset^{cs}$ 93) **where**

$\varphi \supset^{cs} \psi \equiv \lambda V w. \varphi V w \longrightarrow \psi V w$

definition *ValCS* ($\models^{cs} -$) **where** $\models^{cs} \varphi \equiv \forall V w. \varphi V w$

type-synonym $\sigma = \mathcal{W} \Rightarrow \mathcal{R} \Rightarrow \mathcal{R} \Rightarrow \mathcal{V} \Rightarrow \mathcal{U} \Rightarrow \mathcal{E} \Rightarrow \mathcal{W} \Rightarrow bool$

definition *AtmS* :: $\mathcal{S} \Rightarrow \sigma$ ($-^s$ [1000] 999) **where** $x^s \equiv \lambda W B D V U E w. V x w$

definition *DoneS* :: *FatioL* $\Rightarrow \sigma$ (*Done*^s) **where** *Done*^s $l \equiv \lambda W B D V U E w. U l$

w

definition $EntS::Formula \Rightarrow Sign \Rightarrow Formula \Rightarrow \sigma$ **where** $EntS \ \Phi \ sg \ \varphi \equiv \lambda W B D V U E w. E \ \Phi \ sg \ \varphi \ w$

definition $ExJS::Speaker \Rightarrow Sign \Rightarrow Formula \Rightarrow \sigma$ **where**
 $ExJS \ i \ sg \ \varphi \equiv \lambda W B D V U E w. \exists \Delta. U \ (Justify \ i \ \Delta \ sg \ \varphi) \ w$

definition $NegS::\sigma \Rightarrow \sigma$ ($\neg^s$ - [96] 96) **where** $\neg^s \varphi \equiv \lambda W B D V U E w. \neg \varphi \ W B D V U E w$

definition $ImpS::\sigma \Rightarrow \sigma \Rightarrow \sigma$ (**infixr** $\supset^s$ 93) **where**
 $\varphi \supset^s \psi \equiv \lambda W B D V U E w. \varphi \ W B D V U E w \longrightarrow \psi \ W B D V U E w$

definition $BelS::Speaker \Rightarrow \sigma \Rightarrow \sigma$ ($\mathcal{B}^s$) **where**
 $\mathcal{B}^s \ i \ \varphi \equiv \lambda W B D V U E w. \forall v:W. B \ i \ w \ v \longrightarrow \varphi \ W B D V U E v$

definition $DesS::Speaker \Rightarrow \sigma \Rightarrow \sigma$ ($\mathcal{D}^s$) **where**
 $\mathcal{D}^s \ i \ \varphi \equiv \lambda W B D V U E w. \forall v:W. D \ i \ w \ v \longrightarrow \varphi \ W B D V U E v$

definition $RelTS::\mathcal{W} \Rightarrow \mathcal{R} \Rightarrow \mathcal{R} \Rightarrow \mathcal{V} \Rightarrow \mathcal{U} \Rightarrow \mathcal{E} \Rightarrow w \Rightarrow \sigma \Rightarrow bool$ ($\langle -, -, -, -, -, - \rangle, - \models^s -$) **where**
 $\langle W, B, D, V, U, E \rangle, w \models^s \varphi \equiv \varphi \ W B D V U E w$

definition $ValS (\models^s -)$ **where** $\models^s \varphi \equiv \forall W B D V U E. \forall w:W. \langle W, B, D, V, U, E \rangle, w \models^s \varphi$

named-theorems $DefS$

declare $AtmS-def[DefS, simp]$ $DoneS-def[DefS, simp]$ $EntS-def[DefS, simp]$ $ExJS-def[DefS, simp]$
 $NegS-def[DefS, simp]$ $ImpS-def[DefS, simp]$ $BelS-def[DefS, simp]$ $DesS-def[DefS, simp]$
 $RelTS-def[DefS, simp]$ $ValS-def[DefS]$ $AtmCS-def[DefS, simp]$ $NegCS-def[DefS, simp]$
 $ImpCS-def[DefS, simp]$ $ValCS-def[DefS]$

end

4 Shallow embedding (minimal) of the BD logic

theory *FatIoFaithful-shallow-minimal*

imports *FatIoFaithful-deep*

begin

The minimal shallow embedding fixes one model at the meta level: the accessibility relations and valuations are uninterpreted HOL constants, formulas are plain truth sets over worlds, and the world domain is implicitly the full type w . This is the classical SSE style of LogiKEy [3] and the style of the original HOMML/FATIO development; the price, made explicit by the faithfulness theorems in the faithfulness theory, is that faithfulness holds relative to full-domain models only.

consts $B_0::\mathcal{R}$ $D_0::\mathcal{R}$ $V_0::\mathcal{V}$ $U_0::\mathcal{U}$ $E_0::\mathcal{E}$

type-synonym $\sigma_m = w \Rightarrow bool$

definition $AtmM::S \Rightarrow \sigma_m$ ($-^m$ [1000] 999) **where** $x^m \equiv \lambda w. V_0 \ x \ w$

definition $DoneM::FatIoL \Rightarrow \sigma_m$ ($Done^m$) **where** $Done^m \ l \equiv \lambda w. U_0 \ l \ w$

definition $EntM::Formula \Rightarrow Sign \Rightarrow Formula \Rightarrow \sigma_m$ **where** $EntM \ \Phi \ sg \ \varphi \equiv \lambda w. E_0 \ \Phi \ sg \ \varphi \ w$

definition $ExJM::Speaker \Rightarrow Sign \Rightarrow Formula \Rightarrow \sigma_m$ **where**

$ExJM \ i \ sg \ \varphi \equiv \lambda w. \exists \Delta. U_0 \ (Justify \ i \ \Delta \ sg \ \varphi) \ w$

definition $NegM::\sigma_m \Rightarrow \sigma_m \ (\neg^m \cdot \ [96] \ 96)$ **where** $\neg^m \varphi \equiv \lambda w. \neg \varphi \ w$

definition $ImpM::\sigma_m \Rightarrow \sigma_m \Rightarrow \sigma_m$ (**infixr** $\supset^m \ 93$) **where** $\varphi \supset^m \psi \equiv \lambda w. \varphi \ w \longrightarrow \psi \ w$

definition $BelM::Speaker \Rightarrow \sigma_m \Rightarrow \sigma_m \ (\mathcal{B}^m)$ **where** $\mathcal{B}^m \ i \ \varphi \equiv \lambda w. \forall v. B_0 \ i \ w \ v \longrightarrow \varphi \ v$

definition $DesM::Speaker \Rightarrow \sigma_m \Rightarrow \sigma_m \ (\mathcal{D}^m)$ **where** $\mathcal{D}^m \ i \ \varphi \equiv \lambda w. \forall v. D_0 \ i \ w \ v \longrightarrow \varphi \ v$

definition $RelTM::w \Rightarrow \sigma_m \Rightarrow bool \ (- \models^m \cdot)$ **where** $w \models^m \varphi \equiv \varphi \ w$

definition $ValM \ (\models^m \cdot)$ **where** $\models^m \varphi \equiv \forall w. \varphi \ w$

named-theorems $DefM$

declare $AtmM-def[DefM,simp] \ DoneM-def[DefM,simp] \ EntM-def[DefM,simp] \ ExJM-def[DefM,simp]$
 $NegM-def[DefM,simp] \ ImpM-def[DefM,simp] \ BelM-def[DefM,simp] \ DesM-def[DefM,simp]$
 $RelTM-def[DefM,simp] \ ValM-def[DefM]$

end

5 Faithfulness

theory *FatioFaithful-faithfulness*

imports *FatioFaithful-shallow FatioFaithful-shallow-minimal*
begin

5.1 Faithfulness of the content layer

primrec $DpToShC :: Formula \Rightarrow \sigma_c \ (\langle \cdot \rangle^c)$ **where**

$$\langle x^c \rangle^c = x^{cs} \mid \langle \neg^c \varphi \rangle^c = \neg^{cs} \langle \varphi \rangle^c \mid \langle \varphi \supset^c \psi \rangle^c = \langle \varphi \rangle^c \supset^{cs} \langle \psi \rangle^c$$

theorem *Faithful0a*: $\forall V \ w. \ V, w \models^c \varphi \longleftrightarrow \langle \varphi \rangle^c \ V \ w$

apply (*induct* φ) **by** *auto*

theorem *Faithful0b*: $\models^c \varphi \longleftrightarrow \models^{cs} \langle \varphi \rangle^c$

using *Faithful0a* **unfolding** $ValC-def \ ValCS-def$ **by** *auto*

— Mapping: deep to (maximal) shallow

primrec $DpToSh :: BDF \Rightarrow \sigma \ (\llbracket \cdot \rrbracket)$ **where**

$$\begin{aligned} \llbracket x^a \rrbracket &= x^s \mid \llbracket Done^d \ l \rrbracket = Done^s \ l \mid \llbracket EntD \ \Phi \ sg \ \varphi \rrbracket = EntS \ \Phi \ sg \ \varphi \\ \mid \llbracket ExJD \ i \ sg \ \varphi \rrbracket &= ExJS \ i \ sg \ \varphi \mid \llbracket \neg^d \varphi \rrbracket = \neg^s \llbracket \varphi \rrbracket \mid \llbracket \varphi \supset^d \psi \rrbracket = \llbracket \varphi \rrbracket \supset^s \llbracket \psi \rrbracket \\ \mid \llbracket \mathcal{B}^d \ i \ \varphi \rrbracket &= \mathcal{B}^s \ i \ \llbracket \varphi \rrbracket \mid \llbracket \mathcal{D}^d \ i \ \varphi \rrbracket = \mathcal{D}^s \ i \ \llbracket \varphi \rrbracket \end{aligned}$$

— Automated faithfulness proofs, as in [1, 2]

theorem *Faithful1a*:

$$\forall W \ B \ D \ V \ U \ E. \ \forall w:W. \ \langle W, B, D, V, U, E \rangle, w \models^d \varphi \longleftrightarrow \langle W, B, D, V, U, E \rangle, w \models^s \llbracket \varphi \rrbracket$$

apply (*induct* φ) **by** *auto*

theorem *Faithful1b*: $\models^d \varphi \longleftrightarrow \models^s (\llbracket \varphi \rrbracket)$
using *Faithful1a* **unfolding** *ValD-def* *ValS-def* **by** *auto*

5.2 The content layer: faithfulness of its injection into the BD logic

Besides its own deep and shallow embeddings above, the content layer is injected into the BD language by *MapC*, and that injection is truth-preserving. Hence the content layer additionally inherits both shallow embeddings of the BD logic through *MapC*, and its own validity notion coincides with BD validity of its image.

theorem *MapC-faithful*:
 $\langle W, B, D, V, U, E \rangle, w \models^d \{\varphi\}^d \longleftrightarrow V, w \models^c \varphi$
by (*induct* φ) *auto*

theorem *MapC-faithful-shallow*:
assumes $W \ w$
shows $\langle W, B, D, V, U, E \rangle, w \models^s (\llbracket \{\varphi\}^d \rrbracket) \longleftrightarrow V, w \models^c \varphi$
using *assms* *MapC-faithful*[*of* $W \ B \ D \ V \ U \ E \ w \ \varphi$] *Faithful1a*[*of* $\{\varphi\}^d$] **by** *auto*

theorem *MapC-validity*: $(\models^c \varphi) \longleftrightarrow (\models^d \{\varphi\}^d)$
using *MapC-faithful* **unfolding** *ValC-def* *ValD-def* **by** *auto*

5.3 Discrimination vs. identification: the Done-problem, resolved

Syntactically, $p \supset^c p$ and $\neg^c \perp^c$ are different contents; hence locutions and Done-atoms over them are different objects. Semantically, their injections into the BD logic are equivalent. All four facts are theorems of the reconstruction.

lemma *ContentDiscriminated*: $(p^c \supset^c p^c) \neq \neg^c \perp^c$ **by** *simp*

lemma *LocutionDiscriminated*: $\text{assert}[a, p^c \supset^c p^c] \neq \text{assert}[a, \neg^c \perp^c]$ **by** *simp*

lemma *DoneDiscriminated*: $\text{Done}^d \text{assert}[a, p^c \supset^c p^c] \neq \text{Done}^d \text{assert}[a, \neg^c \perp^c]$
by *simp*

lemma *SemanticsIdentifies*:
 $\models^d (\{p^c \supset^c p^c\}^d \supset^d \{\neg^c \perp^c\}^d) \wedge \models^d (\{\neg^c \perp^c\}^d \supset^d \{p^c \supset^c p^c\}^d)$
unfolding *ValD-def* **by** *simp*

5.4 Working on the shallow side: the transfer principle

The maximal shallow embedding is not decoration: consequence in the deep logic can be established entirely by shallow reasoning and transferred back. The following principle is the bridge used by the protocol tests.

theorem *ConsD-via-shallow*:
assumes $\bigwedge W \ B \ D \ V \ U \ E \ w. \ W \ w \implies$
 $(\forall \gamma. \Gamma \ \gamma \longrightarrow (\forall v: W. \langle W, B, D, V, U, E \rangle, v \models^s (\llbracket \gamma \rrbracket)))$

$\implies \langle W, B, D, V, U, E \rangle, w \models^s \langle \varphi \rangle$
shows $\Gamma \models \varphi$
proof (*unfold ConsD-def, intro allI impI*)
fix $W B D V U E w$
assume $\text{sat}: \forall \gamma. \Gamma \gamma \longrightarrow (\forall v: W. \langle W, B, D, V, U, E \rangle, v \models^d \gamma)$ **and** $wW: W w$
have $\text{satS}: \forall \gamma. \Gamma \gamma \longrightarrow (\forall v: W. \langle W, B, D, V, U, E \rangle, v \models^s \langle \gamma \rangle)$
using *sat Faithful1a* **by** *blast*
have $\langle W, B, D, V, U, E \rangle, w \models^s \langle \varphi \rangle$ **using** *shallow[OF wW satS]* .
thus $\langle W, B, D, V, U, E \rangle, w \models^d \varphi$ **using** *wW Faithful1a* **by** *blast*
qed

5.5 The quantifier-scope question, settled

The wide-scope (meta-level) existential of the pre-conditions of question and challenge implies the narrow-scope (object-level, via *ExJD*) reading.

lemma *ScopeImp*:

assumes $\exists \Delta. \Gamma \models \mathcal{D}^d j (\mathcal{B}^d k (\mathcal{D}^d j (\text{Done}^d (\text{Justify } i \Delta \text{ sg } \varphi))))$
shows $\Gamma \models \mathcal{D}^d j (\mathcal{B}^d k (\mathcal{D}^d j (\text{ExJD } i \text{ sg } \varphi)))$
proof –
obtain Δ **where** $H: \Gamma \models \mathcal{D}^d j (\mathcal{B}^d k (\mathcal{D}^d j (\text{Done}^d (\text{Justify } i \Delta \text{ sg } \varphi))))$ **using** *assms* **by** *blast*
show *?thesis* **unfolding** *ConsD-def*
proof (*intro allI impI*)
fix $W B D V U E w$
assume $\text{sat}: \forall \gamma. \Gamma \gamma \longrightarrow (\forall w: W. \langle W, B, D, V, U, E \rangle, w \models^d \gamma)$ **and** $wW: W w$
have $\langle W, B, D, V, U, E \rangle, w \models^d \mathcal{D}^d j (\mathcal{B}^d k (\mathcal{D}^d j (\text{Done}^d (\text{Justify } i \Delta \text{ sg } \varphi))))$
using $H[\text{unfolded ConsD-def}] \text{ sat } wW$ **by** *blast*
thus $\langle W, B, D, V, U, E \rangle, w \models^d \mathcal{D}^d j (\mathcal{B}^d k (\mathcal{D}^d j (\text{ExJD } i \text{ sg } \varphi)))$ **by** *simp blast*
qed
qed

The converse fails, and we refute it AT THE LEVEL OF THE CONSEQUENCE RELATION used by the protocol, not merely pointwise in one model. The refuting situation needs two distinct worlds; since w is only assumed non-empty, this is stated as an explicit hypothesis. The model was found with *nitpick* [5] and is verified here, so that the entry does not depend on the model finder at build time: both worlds carry a justification, but for DIFFERENT supports, so no single Δ works globally.

lemma *ScopeConverse-fails*:

fixes $\varphi_1 \varphi_2 :: \text{Formula}$ **and** $w_1 w_2 :: w$ **and** $i j k :: \text{Speaker}$ **and** $\text{sg} :: \text{Sign}$ **and** $\varphi :: \text{Formula}$
assumes *distinct* $\Delta: \varphi_1 \neq \varphi_2$ **and** *distinct* $w: w_1 \neq w_2$
defines $\Gamma \equiv (\lambda x. x = \mathcal{D}^d j (\mathcal{B}^d k (\mathcal{D}^d j (\text{ExJD } i \text{ sg } \varphi))))$
shows $(\Gamma \models \mathcal{D}^d j (\mathcal{B}^d k (\mathcal{D}^d j (\text{ExJD } i \text{ sg } \varphi))))$
 $\wedge \neg(\exists \Delta. \Gamma \models \mathcal{D}^d j (\mathcal{B}^d k (\mathcal{D}^d j (\text{Done}^d (\text{Justify } i \Delta \text{ sg } \varphi))))$
proof
show $\Gamma \models \mathcal{D}^d j (\mathcal{B}^d k (\mathcal{D}^d j (\text{ExJD } i \text{ sg } \varphi)))$ **unfolding** $\Gamma\text{-def}$ *ConsD-def* **by** *blast*

```

next
  let ?W = λw::w. True
  let ?R = λ(x::Speaker) (w::w) (v::w). True
  let ?U = λl w. l = Justify i (if w = w1 then φ1 else φ2) sg φ
  have prem: ∀γ. Γ γ → (∀w: ?W. ⟨?W, ?R, ?R, V, ?U, E⟩, w ⊨d γ)
    unfolding Γ-def by auto
  show ¬(∃Δ. Γ ⊨ Dd j (Bd k (Dd j (Doned (Justify i Δ sg φ))))))
  proof
    assume ∃Δ. Γ ⊨ Dd j (Bd k (Dd j (Doned (Justify i Δ sg φ))))
    then obtain Δ where
      C: Γ ⊨ Dd j (Bd k (Dd j (Doned (Justify i Δ sg φ)))) by blast
    have ∀w: ?W. ⟨?W, ?R, ?R, V, ?U, E⟩, w ⊨d Dd j (Bd k (Dd j (Doned (Justify i
Δ sg φ))))
      using C[unfolded ConsD-def,
        THEN spec[where x=?W], THEN spec[where x=?R], THEN spec[where
x=?R],
        THEN spec[where x=V], THEN spec[where x=?U], THEN spec[where
x=E]] prem by blast
    hence all: ∧v. ?U (Justify i Δ sg φ) v by simp
    have Δ = φ1 using all[of w1] by simp
    moreover have Δ = φ2 using all[of w2] distinctw by simp
    ultimately show False using distinctΔ by simp
  qed
qed

```

5.6 Faithfulness for the minimal embedding, and the comparison

```

primrec DpToShMin :: BDF ⇒ σm ([·]) where
  [xa] = xm | [Doned l] = Donem l | [EntD Φ sg φ] = EntM Φ sg φ
  | [ExJD i sg φ] = ExJM i sg φ | [¬d φ] = ¬m [φ] | [φ ⊃d ψ] = [φ] ⊃m [ψ]
  | [Bd i φ] = Bm i [φ] | [Dd i φ] = Dm i [φ]

```

Faithfulness for the minimal embedding holds relative to the one fixed, full-domain model given by the meta-level constants.

theorem Faithful2:

```

  ∀w. ⟨(λx::w. True), B0, D0, V0, U0, E0⟩, w ⊨d φ ↔ w ⊨m [φ]
  apply (induct φ) by auto

```

theorem Faithful3:

```

  ∀w. ⟨(λx::w. True), B0, D0, V0, U0, E0⟩, w ⊨s [φ] ↔ w ⊨m [φ]
  using Faithful1a[of φ] Faithful2[of φ] by auto

```

Minimal validity is sound for deep validity in the following precise sense: the minimally valid truth sets are exactly the images of the deeply valid formulas. (This ports lemma Sound1 of FaithfulPMLinHOL, with a structured proof.)

theorem SoundMin: (⊨^m ψ) ↔ (∃φ. ψ = [φ] ∧ ⊨^d φ)

proof

```

assume  $L: \models^m \psi$ 
have  $\psi = \lceil p^a \supset^d p^a \rceil$ 
proof (rule ext)
  fix  $w$  show  $\psi \ w = \lceil p^a \supset^d p^a \rceil \ w$  using  $L$  unfolding  $ValM\text{-}def$  by simp
qed
moreover have  $\models^d (p^a \supset^d p^a)$  unfolding  $ValD\text{-}def$  by simp
ultimately show  $\exists \varphi. \psi = \lceil \varphi \rceil \wedge \models^d \varphi$  by blast
next
assume  $\exists \varphi. \psi = \lceil \varphi \rceil \wedge \models^d \varphi$ 
then obtain  $\varphi$  where  $1: \psi = \lceil \varphi \rceil$  and  $2: \models^d \varphi$  by blast
have  $\forall w. \langle (\lambda x::w. \text{True}), B_0, D_0, V_0, U_0, E_0 \rangle, w \models^d \varphi$  using  $2$  unfolding  $ValD\text{-}def$ 
by simp
thus  $\models^m \psi$  using  $1$  Faithful2 unfolding  $ValM\text{-}def$  by simp
qed

```

Separation of the two shallow backends. Note first a metatheoretical point: an UNCONDITIONAL refutation of "minimal validity implies deep validity" is not available, because minimal validity is decided by the uninterpreted constants B_0, D_0, V_0, U_0, E_0 , about which nothing is provable. The separation is therefore stated conditionally, and this conditionality is itself informative: it says that the minimal embedding cannot even talk about the models in which its validities might fail.

lemma *MinimalNotDeep-conditional:*

```

fixes  $V::\mathcal{V}$  and  $w_1::w$ 
assumes minimally-valid:  $\forall w. V_0 \ x \ w$  and elsewhere-false:  $\neg V \ x \ w_1$ 
shows  $\models^m \lceil x^a \rceil$  and  $\neg \models^d x^a$ 
proof –
  show  $\models^m \lceil x^a \rceil$  using minimally-valid unfolding  $ValM\text{-}def$  by simp
next
show  $\neg \models^d x^a$ 
proof
  assume  $A: \models^d x^a$ 
  have  $V \ x \ w_1$ 
  using  $A[\text{unfolded } ValD\text{-}def,$ 
     $THEN \text{spec}[\text{where } x=\lambda w::w. \text{True}], THEN \text{spec}[\text{where } x=B_0],$ 
     $THEN \text{spec}[\text{where } x=D_0], THEN \text{spec}[\text{where } x=V],$ 
     $THEN \text{spec}[\text{where } x=U_0], THEN \text{spec}[\text{where } x=E_0],$ 
     $THEN \text{spec}[\text{where } x=w_1]]$  by simp
  thus False using elsewhere-false by simp
qed
qed

```

5.7 Modal principles proved shallowly, used deeply

The faithfulness theorems are not decoration: modal principles needed by the protocol are proved on the (maximal) SHALLOW side, where the connectives are plain HOL definitions and the proofs are immediate, and then transferred to the deep syntax over which the protocol is defined. The K

principle for beliefs and desires is obtained this way and is used in the tests theory to discharge a pre-condition that does NOT follow by membership in the world knowledge.

lemma *ShallowKB*: $\models^s (\mathcal{B}^s i (\varphi \supset^s \psi) \supset^s (\mathcal{B}^s i \varphi \supset^s \mathcal{B}^s i \psi))$

unfolding *ValS-def* **by** *auto*

lemma *ShallowKD*: $\models^s (\mathcal{D}^s i (\varphi \supset^s \psi) \supset^s (\mathcal{D}^s i \varphi \supset^s \mathcal{D}^s i \psi))$

unfolding *ValS-def* **by** *auto*

theorem *DeepKB*: $\models^d (\mathcal{B}^d i (\varphi \supset^d \psi) \supset^d (\mathcal{B}^d i \varphi \supset^d \mathcal{B}^d i \psi))$

using *Faithful1b*[*of* $\mathcal{B}^d i (\varphi \supset^d \psi) \supset^d (\mathcal{B}^d i \varphi \supset^d \mathcal{B}^d i \psi)$] *ShallowKB* **by** *simp*

theorem *DeepKD*: $\models^d (\mathcal{D}^d i (\varphi \supset^d \psi) \supset^d (\mathcal{D}^d i \varphi \supset^d \mathcal{D}^d i \psi))$

using *Faithful1b*[*of* $\mathcal{D}^d i (\varphi \supset^d \psi) \supset^d (\mathcal{D}^d i \varphi \supset^d \mathcal{D}^d i \psi)$] *ShallowKD* **by** *simp*

Comparison. The maximal embedding threads the whole model through every formula: validity and the global consequence relation used by the protocol are genuinely model-quantified, faithfulness (*Faithful1a/1b*) is unrestricted, and bounded world domains remain expressible. The minimal embedding is leaner - one world argument instead of seven parameters - and matches the original HOMML/FATIO development; but its faithfulness (*Faithful2*) is relative to the one fixed full-domain model, so dialogue checking on this backend is checking IN a situation rather than a logically necessary entailment. Since the protocol layer is defined over the DEEP syntax, both backends serve it soundly; this entry uses the maximal one for the protocol's consequence relation and provides the minimal one, with the bridging theorems above, for lightweight automation and for continuity with the original sources.

5.8 A worked illustration of the transfer

An illustration that the shallow backend is usable, not merely present: the K principle for the belief operator is proved with the shallow definitions (one call, no induction), and transported to the deep logic by *Faithful1b*. The same route serves any validity needed while checking dialogues.

lemma *K-shallow*: $\models^s (\mathcal{B}^s j (X \supset^s Y) \supset^s (\mathcal{B}^s j X \supset^s \mathcal{B}^s j Y))$

unfolding *ValS-def* **by** *auto*

theorem *K-deep*: $\models^d (\mathcal{B}^d j (\varphi \supset^d \psi) \supset^d (\mathcal{B}^d j \varphi \supset^d \mathcal{B}^d j \psi))$

proof -

have $\models^s (\mathcal{B}^d j (\varphi \supset^d \psi) \supset^d (\mathcal{B}^d j \varphi \supset^d \mathcal{B}^d j \psi))$

using *K-shallow*[*of* $j (\varphi) (\psi)$] **by** *simp*

thus *?thesis* **using** *Faithful1b* **by** *blast*

qed

end

6 The Fatio protocol over the deep syntax

```
theory FatioFaithful-protocol
  imports FatioFaithful-faithfulness
begin
```

6.1 Dialectical obligation stores

```
datatype DOSEntry = Entry Speaker Formula Sign
type-synonym DOS = DOSEntry list
```

```
fun DOSUpdate :: FatioL  $\Rightarrow$  DOS  $\Rightarrow$  DOS where
  DOSUpdate assert[i,  $\varphi$ ] dos = (Entry i  $\varphi \oplus$ ) # dos
  | DOSUpdate question[j, i,  $\varphi$ ] dos = dos
  | DOSUpdate challenge[j, i,  $\varphi$ ] dos = (Entry j  $\varphi \ominus$ ) # dos
  | DOSUpdate (Justify i  $\Phi$  sg  $\varphi$ ) dos = (Entry i  $\Phi$  sg) # dos
  | DOSUpdate retract[i,  $\varphi$ , sg] dos = removeAll (Entry i  $\varphi$  sg) dos
```

6.2 Pre-conditions (axiomatic semantics)

Two reconstruction decisions, both recorded explicitly. (1) Following the Isabelle sources accompanying [7], the justify locution is sign-parametric: $justify[i, \Phi \vdash^\ominus \varphi]$ discharges a negative obligation, as incurred by a challenge. (2) The pre-condition of question is generalised so that the requested justification matches the SIGN of the addressed speaker's obligation. In the original formulation, question requires a POSITIVE obligation and requests a positive justification, which makes it inapplicable to a speaker whose obligation arises from a challenge; the corresponding six-locution dialogue in the example file of those sources is left open there. The generalisation adopted here makes that dialogue checkable (see the tests theory).

```
fun PreCond :: DOS  $\Rightarrow$  (BDF  $\Rightarrow$  bool)  $\Rightarrow$  FatioL  $\Rightarrow$  bool where
  PreCond dos  $\Gamma$  assert[i,  $\varphi$ ] =
    ((Entry i  $\varphi \oplus$ )  $\notin$  set dos  $\wedge$  ( $\forall j. j \neq i \longrightarrow \Gamma \models \mathcal{D}^d i (\mathcal{B}^d j (\mathcal{B}^d i \{\varphi\}^d))$ )))
  | PreCond dos  $\Gamma$  question[j, i,  $\varphi$ ] =
    (i  $\neq j \wedge$  ( $\exists$  sg. (Entry i  $\varphi$  sg)  $\in$  set dos  $\wedge$ 
      ( $\forall k. k \neq j \longrightarrow \Gamma \models \mathcal{D}^d j (\mathcal{B}^d k (\mathcal{D}^d j (ExJD i sg \varphi))))$ )))
  | PreCond dos  $\Gamma$  challenge[j, i,  $\varphi$ ] =
    (i  $\neq j \wedge$  (Entry i  $\varphi \oplus$ )  $\in$  set dos  $\wedge$ 
      ( $\forall k. k \neq j \longrightarrow \Gamma \models \mathcal{D}^d j (\mathcal{B}^d k (\neg^d (\mathcal{B}^d j \{\varphi\}^d)))$ )  $\wedge$ 
      ( $\forall k. k \neq j \longrightarrow \Gamma \models \mathcal{D}^d j (\mathcal{B}^d k (\mathcal{D}^d j (ExJD i \oplus \varphi))))$ )))
  | PreCond dos  $\Gamma$  (Justify i  $\Phi$  sg  $\varphi$ ) =
    ((Entry i  $\varphi$  sg)  $\in$  set dos  $\wedge$ 
      ( $\exists j. j \neq i \wedge$  ( $\Gamma$  (Doned question[j, i,  $\varphi$ ])  $\vee$   $\Gamma$  (Doned challenge[j, i,  $\varphi$ ]))  $\wedge$ 
      ( $\forall k. k \neq i \longrightarrow \Gamma \models \mathcal{D}^d i (\mathcal{B}^d k (\mathcal{B}^d i (EntD \Phi sg \varphi))))$ )))
  | PreCond dos  $\Gamma$  retract[i,  $\varphi$ , sg] =
    ((Entry i  $\varphi$  sg)  $\in$  set dos  $\wedge$ 
      (case sg of  $\oplus \Rightarrow$  ( $\forall j. j \neq i \longrightarrow \Gamma \models \mathcal{D}^d i (\mathcal{B}^d j (\neg^d (\mathcal{B}^d i \{\varphi\}^d)))$ )
        |  $\ominus \Rightarrow$  ( $\forall j. j \neq i \longrightarrow \Gamma \models \mathcal{D}^d i (\mathcal{B}^d j (\neg^d \neg^d (\mathcal{B}^d i \{\varphi\}^d)))$ )))
```

6.3 Post-conditions and state evolution

fun *GammaAdd* :: *FatL* $\Rightarrow$ (*BDF* $\Rightarrow$ *bool*) **where**
 GammaAdd assert[*i*, φ] =
 $(\lambda x. \exists k j. k \neq i \wedge j \neq i \wedge x = \mathcal{B}^d k (\mathcal{D}^d i (\mathcal{B}^d j (\mathcal{B}^d i \{\varphi\}^d))))$
 | *GammaAdd question*[*j*,*i*, φ] =
 $(\lambda x. \exists k sg. k \neq j \wedge x = \mathcal{B}^d k (\mathcal{D}^d j (ExJD\ i\ sg\ \varphi)))$
 | *GammaAdd challenge*[*j*,*i*, φ] =
 $(\lambda x. \exists k. k \neq j \wedge (x = \mathcal{B}^d k (\mathcal{D}^d j (ExJD\ i \oplus \varphi))$
 $\vee x = \mathcal{B}^d k (\mathcal{D}^d j (\neg^d (\mathcal{B}^d j \{\varphi\}^d))))$
 | *GammaAdd (Justify i Φ sg φ)* =
 $(\lambda x. \exists k j. k \neq i \wedge j \neq i \wedge x = \mathcal{B}^d k (\mathcal{D}^d i (\mathcal{B}^d j (\mathcal{B}^d i (EntD\ \Phi\ sg\ \varphi))))$
 | *GammaAdd retract*[*i*, φ ,*sg*] =
 $(case\ sg\ of\ \oplus \Rightarrow (\lambda x. \exists k j. k \neq i \wedge j \neq i \wedge x = \mathcal{B}^d k (\mathcal{D}^d i (\mathcal{B}^d j (\neg^d (\mathcal{B}^d i$
 $\{\varphi\}^d))))$
 $\mid \ominus \Rightarrow (\lambda x. \exists k j. k \neq i \wedge j \neq i \wedge x = \mathcal{B}^d k (\mathcal{D}^d i (\mathcal{B}^d j (\neg^d \neg^d (\mathcal{B}^d i$
 $\{\varphi\}^d))))))$

The state update filters: a previously held formula survives unless the locution asserts its negation. The earlier implementation contained such a filter but left it deactivated, updating by plain union; that variant, and a theorem exhibiting a state on which the two differ, are recorded in the variants theory. Filtering matters because an inconsistent state trivialises every precondition (see *ClashTrivialises* there); independently of it, the satisfiability of the world knowledge used in the tests theory is verified there.

definition *GammaKeep* :: *FatL* $\Rightarrow$ (*BDF* $\Rightarrow$ *bool*) **where**

GammaKeep l $\equiv \lambda x. \neg(\exists y. (GammaAdd\ l\ y \vee y = Done^d\ l) \wedge x = \neg^d y)$

— A formula can only be filtered out if it is a negation; all other shapes survive every update, which discharges the side conditions automatically.

lemma *GammaKeep-AtmD[simp]*: *GammaKeep l* (x^a) **unfolding** *GammaKeep-def* **by** *simp*

lemma *GammaKeep-DoneD[simp]*: *GammaKeep l* ($Done^d\ l'$) **unfolding** *GammaKeep-def* **by** *simp*

lemma *GammaKeep-EntD[simp]*: *GammaKeep l* ($EntD\ \Phi\ sg\ \varphi$) **unfolding** *GammaKeep-def* **by** *simp*

lemma *GammaKeep-ExJD[simp]*: *GammaKeep l* ($ExJD\ i\ sg\ \varphi$) **unfolding** *GammaKeep-def* **by** *simp*

lemma *GammaKeep-ImpD[simp]*: *GammaKeep l* ($\varphi \supset^d \psi$) **unfolding** *GammaKeep-def* **by** *simp*

lemma *GammaKeep-BelD[simp]*: *GammaKeep l* ($\mathcal{B}^d\ i\ \varphi$) **unfolding** *GammaKeep-def* **by** *simp*

lemma *GammaKeep-DesD[simp]*: *GammaKeep l* ($\mathcal{D}^d\ i\ \varphi$) **unfolding** *GammaKeep-def* **by** *simp*

definition *GammaUpdate* :: *FatL* $\Rightarrow$ (*BDF* $\Rightarrow$ *bool*) $\Rightarrow$ (*BDF* $\Rightarrow$ *bool*) **where**

GammaUpdate l $\Gamma \equiv \lambda x. (\Gamma\ x \wedge GammaKeep\ l\ x) \vee GammaAdd\ l\ x \vee x = Done^d\ l$

— Checking a dialogue: each locution must meet its pre-condition in the state reached so far, and updates store and world knowledge for the rest.

fun *FatioCheckRec* :: *FatioL list* $\Rightarrow$ *DOS* $\Rightarrow$ (*BDF* $\Rightarrow$ *bool*) $\Rightarrow$ *bool* **where**
 FatioCheckRec [] *dos* Γ = *True*
 | *FatioCheckRec* (*l* # *ls*) *dos* Γ =
 (*PreCond dos* Γ *l* $\wedge$ *FatioCheckRec ls* (*DOSUpdate l dos*) (*GammaUpdate l*
 Γ))

6.4 Outstanding requests (Fatio's obligation to respond)

A question or a challenge obliges the addressee to respond, by justifying or by retracting. The earlier formalisation left this to informal combination rules; here it is made checkable: *Pending* collects the requests that are still open after a sequence of locutions, and a dialogue is *Answered* when none remain.

fun *Pending* :: *FatioL list* $\Rightarrow$ (*Speaker* $\times$ *Formula*) *set* $\Rightarrow$ (*Speaker* $\times$ *Formula*) *set* **where**
 Pending [] *P* = *P*
 | *Pending* (*assert*[*i*, φ] # *ls*) *P* = *Pending ls P*
 | *Pending* (*question*[*j*,*i*, φ] # *ls*) *P* = *Pending ls* (*insert* (*i*, φ) *P*)
 | *Pending* (*challenge*[*j*,*i*, φ] # *ls*) *P* = *Pending ls* (*insert* (*i*, φ) *P*)
 | *Pending* (*Justify i* Φ *sg* φ # *ls*) *P* = *Pending ls* (*P* - {(*i*, φ)})
 | *Pending* (*retract*[*i*, φ ,*sg*] # *ls*) *P* = *Pending ls* (*P* - {(*i*, φ)})

definition *Answered* :: *FatioL list* $\Rightarrow$ *bool* **where** *Answered ls* $\equiv$ *Pending ls* {} = {}

lemma *PendingRequest*: *Pending* [*question*[*j*,*i*, φ]] {} = {(*i*, φ)} **by** *simp*

lemma *JustifyAnswers*: *Answered* [*question*[*j*,*i*, φ], *Justify i* Φ *sg* φ]

unfolding *Answered-def* **by** *simp*

lemma *RetractAnswers*: *Answered* [*challenge*[*j*,*i*, φ], *retract*[*i*, φ ,*sg*]]

unfolding *Answered-def* **by** *simp*

lemma *UnansweredChallenge*: $\neg$ *Answered* [*assert*[*i*, φ], *challenge*[*j*,*i*, φ]]

unfolding *Answered-def* **by** *simp*

6.5 Sanity theorems

lemma *ConsKD $\mathcal{B}$* :

assumes $\Gamma \models \mathcal{D}^d i (\mathcal{B}^d j (\varphi \supset^d \psi))$ **and** $\Gamma \models \mathcal{D}^d i (\mathcal{B}^d j \varphi)$

shows $\Gamma \models \mathcal{D}^d i (\mathcal{B}^d j \psi)$

using *assms* **unfolding** *ConsD-def* **by** *simp*

lemma *MemberEntails[intro]*: $\Gamma \varphi \implies \Gamma \models \varphi$ **unfolding** *ConsD-def* **by** *blast*

— Consequence is more than membership: the modal K principle for the desire and belief operators lets a pre-condition follow from world knowledge that does not contain it literally. This is used in the tests theory.

lemma *ConsK*:

assumes $X: \Gamma (\mathcal{D}^d i (\mathcal{B}^d j X))$ **and** $XY: \Gamma (\mathcal{D}^d i (\mathcal{B}^d j (X \supset^d Y)))$

shows $\Gamma \models \mathcal{D}^d i (\mathcal{B}^d j Y)$

```

proof (unfold ConsD-def, intro allI impI)
  fix W B D V U E w
  assume sat:  $\forall \gamma. \Gamma \gamma \longrightarrow (\forall w: W. \langle W, B, D, V, U, E \rangle, w \models^d \gamma)$  and wW:  $W w$ 
  have h1:  $\langle W, B, D, V, U, E \rangle, w \models^d \mathcal{D}^d i (\mathcal{B}^d j X)$  using sat X wW by blast
  have h2:  $\langle W, B, D, V, U, E \rangle, w \models^d \mathcal{D}^d i (\mathcal{B}^d j (X \supset^d Y))$  using sat XY wW by
blast
  show  $\langle W, B, D, V, U, E \rangle, w \models^d \mathcal{D}^d i (\mathcal{B}^d j Y)$  using h1 h2 by auto
qed

```

— With the filter active, monotonicity is conditional: a formula survives an update unless the locution asserts its negation.

```

lemma GammaMono:  $\llbracket \Gamma x; \text{GammaKeep } l \ x \rrbracket \Longrightarrow \text{GammaUpdate } l \ \Gamma \ x$ 
  by (simp add: GammaUpdate-def)
lemma DoneRecorded:  $\text{GammaUpdate } l \ \Gamma \ (\text{Done}^d \ l)$  by (simp add: GammaUpdate-def)
lemma AssertCreatesObligation:  $(\text{Entry } i \ \varphi \oplus) \in \text{set } (\text{DOSUpdate } \text{assert}[i, \varphi] \ \text{dos})$ 
by simp
lemma RetractRemovesObligation:  $(\text{Entry } i \ \varphi \ \text{sg}) \notin \text{set } (\text{DOSUpdate } \text{retract}[i, \varphi, \text{sg}] \ \text{dos})$ 
by simp
end

```

7 Variants: the earlier formalisation, and an optional argumentative bridge

```

theory FatioFaithful-variants
  imports FatioFaithful-protocol
begin

```

The reconstruction restores several features that the earlier formalisation [7] had simplified away: the sign-sensitive pre-condition of question, the consistency filter on the state update, and the post-conditions of question and challenge. Since removing those simplifications changes the object of study, the earlier treatments are recorded here, so that the difference is visible and checkable rather than merely asserted.

7.1 Variant: the original, sign-blind question pre-condition

For comparison, the pre-condition of question exactly as in the sources accompanying [7]: it requires a POSITIVE obligation of the addressed speaker and requests a POSITIVE justification. All other clauses are unchanged, so the two protocols differ only in this one respect.

```

fun PreCondO ::  $\text{DOS} \Rightarrow (\text{BDF} \Rightarrow \text{bool}) \Rightarrow \text{FatioL} \Rightarrow \text{bool}$  where
  PreCondO dos  $\Gamma \ \text{question}[j, i, \varphi] =$ 
     $(i \neq j \wedge (\text{Entry } i \ \varphi \oplus) \in \text{set } \text{dos} \wedge$ 
       $(\forall k. k \neq j \longrightarrow \Gamma \models \mathcal{D}^d j (\mathcal{B}^d k (\mathcal{D}^d j (\text{ExJD } i \oplus \varphi))))$ 
    | PreCondO dos  $\Gamma \ l = \text{PreCond } \text{dos } \Gamma \ l$ 

```



```

fun FatioCheckRecO :: FatioL list  $\Rightarrow$  DOS  $\Rightarrow$  (BDF  $\Rightarrow$  bool)  $\Rightarrow$  bool where
  FatioCheckRecO [] dos  $\Gamma$  = True
  | FatioCheckRecO (l # ls) dos  $\Gamma$  =
    (PreCondO dos  $\Gamma$  l  $\wedge$  FatioCheckRecO ls (DOSUpdate l dos) (GammaUpdate
l  $\Gamma$ ))

```

The two agree except on question, and on question they agree whenever the addressed obligation is positive; they differ exactly when it is negative, i.e. when the question addresses a speaker who has challenged.

```

lemma PreCondO-eq-off-question:
  assumes  $\bigwedge j\ i\ \varphi.\ l \neq \text{question}[j,i,\varphi]$ 
  shows PreCondO dos  $\Gamma$  l = PreCond dos  $\Gamma$  l
  using assms by (cases l) auto

```

```

lemma PreCondO-blocks-negative:
  assumes (Entry i  $\varphi \oplus$ )  $\notin$  set dos
  shows  $\neg$  PreCondO dos  $\Gamma$  question[j,i, $\varphi$ ]
  using assms by simp

```

7.2 Variant: the earlier, unfiltered state update

The earlier formalisation updated the state by plain union, with the filter present but deactivated. That variant is recorded here, together with the reason the filter is active in the reconstruction: an inconsistent state trivialises every pre-condition.

```

lemma ClashTrivialises:
  assumes  $\Gamma\ \psi$  and  $\Gamma\ (\neg^d \psi)$ 
  shows  $\Gamma \models \varphi$ 
proof (unfold ConsD-def, intro allI impI)
  fix W B D V U E w
  assume sat:  $\forall \gamma.\ \Gamma\ \gamma \longrightarrow (\forall w:W.\ \langle W,B,D,V,U,E \rangle, w \models^d \gamma)$  and wW: W w
  have  $\langle W,B,D,V,U,E \rangle, w \models^d \psi$  using sat assms(1) wW by blast
  moreover have  $\langle W,B,D,V,U,E \rangle, w \models^d \neg^d \psi$  using sat assms(2) wW by blast
  ultimately show  $\langle W,B,D,V,U,E \rangle, w \models^d \varphi$  by simp
qed

```

```

definition GammaUpdateM :: FatioL  $\Rightarrow$  (BDF  $\Rightarrow$  bool)  $\Rightarrow$  (BDF  $\Rightarrow$  bool) where
  GammaUpdateM l  $\Gamma$   $\equiv$   $\lambda x.\ \Gamma\ x \vee \text{GammaAdd } l\ x \vee x = \text{Done}^d\ l$ 

```

```

lemma GammaUpdate-weaker: GammaUpdate l  $\Gamma\ x \implies \text{GammaUpdateM } l\ \Gamma\ x$ 
  unfolding GammaUpdate-def GammaUpdateM-def by blast

```

```

lemma UpdatesAgree:
  assumes  $\bigwedge x.\ \Gamma\ x \implies \text{GammaKeep } l\ x$ 
  shows GammaUpdate l  $\Gamma\ x \longleftrightarrow \text{GammaUpdateM } l\ \Gamma\ x$ 
  using assms unfolding GammaUpdate-def GammaUpdateM-def by blast

```

lemma *UnfilteredKeepsClash*:
fixes $i::\text{Speaker}$ **and** $\varphi::\text{Formula}$
assumes $\Gamma_d = (\lambda x. x = \neg^d(\text{Done}^d \text{assert}[i, \varphi]))$
shows $\text{GammaUpdateM } \text{assert}[i, \varphi] \Gamma_d (\text{Done}^d \text{assert}[i, \varphi])$
 $\wedge \text{GammaUpdateM } \text{assert}[i, \varphi] \Gamma_d (\neg^d(\text{Done}^d \text{assert}[i, \varphi]))$
using *assms* **unfolding** *GammaUpdateM-def* **by** *simp*

lemma *FilteredDropsClash*:
fixes $i::\text{Speaker}$ **and** $\varphi::\text{Formula}$
assumes $\Gamma_d = (\lambda x. x = \neg^d(\text{Done}^d \text{assert}[i, \varphi]))$
shows $\text{GammaUpdate } \text{assert}[i, \varphi] \Gamma_d (\text{Done}^d \text{assert}[i, \varphi])$
 $\wedge \neg \text{GammaUpdate } \text{assert}[i, \varphi] \Gamma_d (\neg^d(\text{Done}^d \text{assert}[i, \varphi]))$
using *assms* **unfolding** *GammaUpdate-def* *GammaKeep-def* **by** *auto*

7.3 Optional bridge: entailment atoms with content

In both the sources and the reconstruction the entailment atoms are interpreted by an arbitrary valuation: nothing relates $\text{EntD } \Phi \oplus \varphi$, read as "the support is an argument for the claim", to the content logic. Now that the content layer has a semantics, the natural constraint can be stated: a model is *ArgSound* if every supported claim is entailed by its support, and every attacked claim is refuted by it.

definition $\text{CEnt} :: \text{Formula} \Rightarrow \text{Formula} \Rightarrow \mathcal{V} \Rightarrow \text{bool}$ **where**
 $\text{CEnt } \Phi \varphi V \equiv \forall w. V, w \models^c \Phi \longrightarrow V, w \models^c \varphi$

definition $\text{ArgSound} :: \mathcal{V} \Rightarrow \mathcal{E} \Rightarrow \text{bool}$ **where**
 $\text{ArgSound } V E \equiv \forall \Phi \varphi w. (E \Phi \oplus \varphi w \longrightarrow \text{CEnt } \Phi \varphi V)$
 $\wedge (E \Phi \ominus \varphi w \longrightarrow \text{CEnt } \Phi (\neg^c \varphi) V)$

lemma *ArgSound-nontrivial*: $\text{ArgSound } V (\lambda \Phi \text{ sg } \varphi w. \text{False})$
unfolding *ArgSound-def* **by** *simp*

lemma *ArgSound-canonical*: $\text{ArgSound } V (\lambda \Phi \text{ sg } \varphi w. \text{case sg of } \oplus \Rightarrow \text{CEnt } \Phi \varphi V$
 $\mid \ominus \Rightarrow \text{CEnt } \Phi (\neg^c \varphi) V)$

unfolding *ArgSound-def* **by** *simp*

In an *ArgSound* model a justification does what its name suggests: whoever grants the support must grant the claim. Without the constraint this fails.

theorem *JustificationTransfers*:
assumes *sound*: $\text{ArgSound } V E$
and *just*: $\langle W, B, D, V, U, E \rangle, w \models^d \text{EntD } \Phi \oplus \varphi$
and *support*: $\langle W, B, D, V, U, E \rangle, w \models^d \{\Phi\}^d$
shows $\langle W, B, D, V, U, E \rangle, w \models^d \{\varphi\}^d$
proof –
have $\text{CEnt } \Phi \varphi V$ **using** *sound* *just* **unfolding** *ArgSound-def* **by** *simp*
moreover **have** $V, w \models^c \Phi$ **using** *support* *MapC-faithful* **by** *blast*

ultimately have $V, w \models^c \varphi$ **unfolding** *CEnt-def* **by** *blast*
 thus *?thesis* **using** *MapC-faithful* **by** *blast*
qed

theorem *WithoutBridgeArbitrary*:
 fixes $x y :: \mathcal{S}$ and $V :: \mathcal{V}$
 assumes $V x w$ and $\neg V y w$
 shows $\langle W, B, D, V, U, (\lambda \Phi \text{ sg } \varphi \text{ w. True}) \rangle, w \models^d \text{EntD } (x^c) \oplus (y^c)$
 $\wedge \neg \text{CEnt } (x^c) (y^c) V$
 using *assms* **unfolding** *CEnt-def* **by** *auto*

The bridge is offered, not imposed, and the reason is visible in the running example: the newspaper review n and the quality of the restaurant r are distinct atoms, so n does not entail r in the content logic. Under *ArgSound* the justification of the Brigade dialogue would therefore be unavailable - the support of a real argument is defeasible, not deductive. Keeping the entailment atoms unconstrained is thus not an oversight of the original formalisation but a commitment to defeasible support; *ArgSound* records what the deductive reading would cost.

theorem *DefeasibleSupportNeeded*:
 fixes $V :: \mathcal{V}$ and $n r :: \mathcal{S}$
 assumes $V n w$ and $\neg V r w$
 shows $\neg \text{CEnt } (n^c) (r^c) V$
 using *assms* **unfolding** *CEnt-def* **by** *auto*

end

8 Tests: two dialogues, checked with high automation

theory *FatioFaithful-tests*
 imports *FatioFaithful-variants*
begin

— example content atoms (as in the Fatio paper)
consts $p :: \mathcal{S}$ $q :: \mathcal{S}$ $r :: \mathcal{S}$ $n :: \mathcal{S}$ $m :: \mathcal{S}$ $s :: \mathcal{S}$

8.1 The full Brigade Restaurant dialogue (from the literature)

The six-locution Brigade Restaurant dialogue. It appears in the example file of the Isabelle sources accompanying [7], attributed there to the original Fatio paper [6]: a asserts that the restaurant is good (r); b challenges; a justifies from the newspaper review (n); c questions b, whose obligation is NEGATIVE, having arisen from the challenge; b justifies negatively from the chef change and the quality drop ($m \wedge^c s$); finally a retracts. In the original

sources the corresponding lemma is stated with empty world knowledge and left open (with a counterexample noted); here the initial knowledge Γ_B supplies exactly the mental-state formulas that the pre-conditions require, and the sign-generalised question pre-condition (see the protocol theory) makes the fourth locution applicable, so that the dialogue checks.

definition $\Gamma_B :: BDF \Rightarrow \text{bool}$ **where** $\Gamma_B \equiv \lambda x.$

$$\begin{aligned} & (\exists j. j \neq a \wedge x = \mathcal{D}^d a (\mathcal{B}^d j (\mathcal{B}^d a \{r^c\}^d))) \\ \vee & (\exists k. k \neq b \wedge x = \mathcal{D}^d b (\mathcal{B}^d k (\neg^d (\mathcal{B}^d b \{r^c\}^d)))) \\ \vee & (\exists k. k \neq b \wedge x = \mathcal{D}^d b (\mathcal{B}^d k (\mathcal{D}^d b (ExJD a \oplus (r^c))))) \\ \vee & (\exists k. k \neq a \wedge x = \mathcal{D}^d a (\mathcal{B}^d k (\mathcal{B}^d a (EntD (n^c) \oplus (r^c))))) \\ \vee & (\exists k. k \neq c \wedge x = \mathcal{D}^d c (\mathcal{B}^d k (\mathcal{D}^d c (ExJD b \ominus (r^c))))) \\ \vee & (\exists k. k \neq b \wedge x = \mathcal{D}^d b (\mathcal{B}^d k (\mathcal{B}^d b (EntD (m^c \wedge^c s^c) \ominus (r^c))))) \\ \vee & (\exists j. j \neq a \wedge x = \mathcal{D}^d a (\mathcal{B}^d j (\neg^d (\mathcal{B}^d a \{r^c\}^d)))) \end{aligned}$$

abbreviation *Brigade* $\equiv$

$$\begin{aligned} & [\text{assert}[a, r^c], \text{challenge}[b, a, r^c], \text{justify}[a, n^c \vdash^\oplus r^c], \\ & \text{question}[c, b, r^c], \text{justify}[b, m^c \wedge^c s^c \vdash^\ominus r^c], \text{retract}[a, r^c, \oplus]] \end{aligned}$$

Automation. Every step obligation is discharged by a single automated call (*auto* with the two protocol definitions and *MemberEntails* as an introduction rule; the side conditions created by the state filter are closed by the *GammaKeep* simplification rules of the protocol theory). The justify steps additionally exhibit their Done-witness in two lines beforehand, and the question steps select the sign of the addressed obligation. The dialogues themselves are then assembled from the step lemmas without search. This division of labour is deliberate: *sledgehammer* [4] finds no proof for these steps even with generous time limits and the relevant facts supplied, because what they require is the choice of a WITNESS over a finite datatype (which speaker questioned or challenged, which sign the obligation carries) rather than a derivation; automated provers are strong at the latter and weak at the former. Supplying the witness explicitly and automating the rest is therefore not a workaround but the appropriate split.

abbreviation $dos_1 \equiv DOSUpdate \text{assert}[a, r^c] []$

abbreviation $dos_2 \equiv DOSUpdate \text{challenge}[b, a, r^c] dos_1$

abbreviation $dos_3 \equiv DOSUpdate \text{justify}[a, n^c \vdash^\oplus r^c] dos_2$

abbreviation $dos_4 \equiv DOSUpdate \text{question}[c, b, r^c] dos_3$

abbreviation $dos_5 \equiv DOSUpdate \text{justify}[b, m^c \wedge^c s^c \vdash^\ominus r^c] dos_4$

abbreviation $\Gamma_1 \equiv GammaUpdate \text{assert}[a, r^c] \Gamma_B$

abbreviation $\Gamma_2 \equiv GammaUpdate \text{challenge}[b, a, r^c] \Gamma_1$

abbreviation $\Gamma_3 \equiv GammaUpdate \text{justify}[a, n^c \vdash^\oplus r^c] \Gamma_2$

abbreviation $\Gamma_4 \equiv GammaUpdate \text{question}[c, b, r^c] \Gamma_3$

abbreviation $\Gamma_5 \equiv GammaUpdate \text{justify}[b, m^c \wedge^c s^c \vdash^\ominus r^c] \Gamma_4$

lemma *B1: PreCond* $[] \Gamma_B \text{assert}[a, r^c]$

by (*auto simp: Γ_B -def intro!: MemberEntails*)

lemma *B2: PreCond* $dos_1 \Gamma_1 \text{challenge}[b, a, r^c]$

by (*auto simp: Γ_B -def GammaUpdate-def intro!: MemberEntails*)

lemma *B3*: $PreCond\ dos_2\ \Gamma_2\ justify[a, n^c \vdash^\oplus r^c]$
proof –
 have $\Gamma_2\ (Done^d\ challenge[b, a, r^c])$ **by** (*simp add: GammaUpdate-def*)
 hence $\exists j. j \neq a \wedge (\Gamma_2\ (Done^d\ question[j, a, r^c]) \vee \Gamma_2\ (Done^d\ challenge[j, a, r^c]))$
 by (*intro exI[of - b] simp*)
 thus *?thesis* **by** (*auto simp: Γ_B -def GammaUpdate-def intro!: MemberEntails*)
qed
lemma *B4*: $PreCond\ dos_3\ \Gamma_3\ question[c, b, r^c]$
 by (*intro PreCond.simps[THEN iffD2] conjI exI[of - $\ominus$]*)
 (*auto simp: Γ_B -def GammaUpdate-def intro!: MemberEntails*)
lemma *B5*: $PreCond\ dos_4\ \Gamma_4\ justify[b, m^c \wedge^c s^c \vdash^\ominus r^c]$
proof –
 have $\Gamma_4\ (Done^d\ question[c, b, r^c])$ **by** (*simp add: GammaUpdate-def*)
 hence $\exists j. j \neq b \wedge (\Gamma_4\ (Done^d\ question[j, b, r^c]) \vee \Gamma_4\ (Done^d\ challenge[j, b, r^c]))$
 by (*intro exI[of - c] simp*)
 thus *?thesis* **by** (*auto simp: Γ_B -def GammaUpdate-def intro!: MemberEntails*)
qed
lemma *B6*: $PreCond\ dos_5\ \Gamma_5\ retract[a, r^c, \oplus]$
 by (*auto simp: Γ_B -def GammaUpdate-def intro!: MemberEntails*)

theorem *BrigadeDialogue*: $FatioCheckRec\ Brigade\ []\ \Gamma_B$
unfolding *FatioCheckRec.simps*
by (*intro conjI TrueI*) (*fact B1, fact B2, fact B3, fact B4, fact B5, fact B6*)

8.2 A further example, constructed here: role reversal with a negative retraction

Seven locutions over two topics: after the exchange on p is settled by b RETRACTING ITS NEGATIVE obligation (exercising the $\ominus$ -clause of retract), the roles reverse and b becomes theasserter of q , questioned by a .

definition $\Gamma_C :: BDF \Rightarrow bool$ **where** $\Gamma_C \equiv \lambda x.$
 $(\exists j. j \neq a \wedge x = \mathcal{D}^d\ a\ (\mathcal{B}^d\ j\ (\mathcal{B}^d\ a\ \{p^c\}^d)))$
 $\vee (\exists k. k \neq b \wedge x = \mathcal{D}^d\ b\ (\mathcal{B}^d\ k\ (\neg^d(\mathcal{B}^d\ b\ \{p^c\}^d))))$
 $\vee (\exists k. k \neq b \wedge x = \mathcal{D}^d\ b\ (\mathcal{B}^d\ k\ (\mathcal{D}^d\ b\ (ExJD\ a\ \oplus\ (p^c))))))$
 $\vee (\exists k. k \neq a \wedge x = \mathcal{D}^d\ a\ (\mathcal{B}^d\ k\ (\mathcal{B}^d\ a\ (EntD\ (n^c) \oplus\ (p^c))))))$
 $\vee (\exists j. j \neq b \wedge x = \mathcal{D}^d\ b\ (\mathcal{B}^d\ j\ (\neg^d\neg^d(\mathcal{B}^d\ b\ \{p^c\}^d))))$
 $\vee (\exists j. j \neq b \wedge x = \mathcal{D}^d\ b\ (\mathcal{B}^d\ j\ (\mathcal{B}^d\ b\ \{q^c\}^d)))$
 $\vee (\exists k. k \neq a \wedge x = \mathcal{D}^d\ a\ (\mathcal{B}^d\ k\ (\mathcal{D}^d\ a\ (ExJD\ b\ \oplus\ (q^c))))))$
 $\vee (\exists k. k \neq b \wedge x = \mathcal{D}^d\ b\ (\mathcal{B}^d\ k\ (\mathcal{B}^d\ b\ (EntD\ (m^c) \oplus\ (q^c))))))$

abbreviation *Cascade* $\equiv$
 $[assert[a, p^c], challenge[b, a, p^c], justify[a, n^c \vdash^\oplus p^c], retract[b, p^c, \ominus],$
 $assert[b, q^c], question[a, b, q^c], justify[b, m^c \vdash^\oplus q^c]]$

abbreviation $cd_1 \equiv DOSUpdate\ assert[a, p^c]\ []$
abbreviation $cd_2 \equiv DOSUpdate\ challenge[b, a, p^c]\ cd_1$
abbreviation $cd_3 \equiv DOSUpdate\ justify[a, n^c \vdash^\oplus p^c]\ cd_2$
abbreviation $cd_4 \equiv DOSUpdate\ retract[b, p^c, \ominus]\ cd_3$
abbreviation $cd_5 \equiv DOSUpdate\ assert[b, q^c]\ cd_4$

abbreviation $cd_6 \equiv DOSUpdate\ question[a, b, q^c]\ cd_5$
abbreviation $\Delta_1 \equiv GammaUpdate\ assert[a, p^c]\ \Gamma_C$
abbreviation $\Delta_2 \equiv GammaUpdate\ challenge[b, a, p^c]\ \Delta_1$
abbreviation $\Delta_3 \equiv GammaUpdate\ justify[a, n^c \vdash^\oplus p^c]\ \Delta_2$
abbreviation $\Delta_4 \equiv GammaUpdate\ retract[b, p^c, \ominus]\ \Delta_3$
abbreviation $\Delta_5 \equiv GammaUpdate\ assert[b, q^c]\ \Delta_4$
abbreviation $\Delta_6 \equiv GammaUpdate\ question[a, b, q^c]\ \Delta_5$

lemma $C1$: $PreCond\ []\ \Gamma_C\ assert[a, p^c]$
by (*auto simp: Γ_C -def intro!: MemberEntails*)
lemma $C2$: $PreCond\ cd_1\ \Delta_1\ challenge[b, a, p^c]$
by (*auto simp: Γ_C -def GammaUpdate-def intro!: MemberEntails*)
lemma $C3$: $PreCond\ cd_2\ \Delta_2\ justify[a, n^c \vdash^\oplus p^c]$
proof –
have $\Delta_2\ (Done^d\ challenge[b, a, p^c])$ **by** (*simp add: GammaUpdate-def*)
hence $\exists j. j \neq a \wedge (\Delta_2\ (Done^d\ question[j, a, p^c]) \vee \Delta_2\ (Done^d\ challenge[j, a, p^c]))$
by (*intro exI[of - b] simp*)
thus *?thesis* **by** (*auto simp: Γ_C -def GammaUpdate-def intro!: MemberEntails*)
qed
lemma $C4$: $PreCond\ cd_3\ \Delta_3\ retract[b, p^c, \ominus]$
by (*auto simp: Γ_C -def GammaUpdate-def intro!: MemberEntails*)
lemma $C5$: $PreCond\ cd_4\ \Delta_4\ assert[b, q^c]$
by (*auto simp: Γ_C -def GammaUpdate-def intro!: MemberEntails*)
lemma $C6$: $PreCond\ cd_5\ \Delta_5\ question[a, b, q^c]$
by (*intro PreCond.simps[THEN iffD2] conjI exI[of - $\oplus$]*)
(auto simp: Γ_C -def GammaUpdate-def intro!: MemberEntails)
lemma $C7$: $PreCond\ cd_6\ \Delta_6\ justify[b, m^c \vdash^\oplus q^c]$
proof –
have $\Delta_6\ (Done^d\ question[a, b, q^c])$ **by** (*simp add: GammaUpdate-def*)
hence $\exists j. j \neq b \wedge (\Delta_6\ (Done^d\ question[j, b, q^c]) \vee \Delta_6\ (Done^d\ challenge[j, b, q^c]))$
by (*intro exI[of - a] simp*)
thus *?thesis* **by** (*auto simp: Γ_C -def GammaUpdate-def intro!: MemberEntails*)
qed

theorem *CascadeDialogue*: *FatioCheckRec Cascade* $[]\ \Gamma_C$
unfolding *FatioCheckRec.simps*
by (*intro conjI TrueI*) (*fact C1, fact C2, fact C3, fact C4, fact C5, fact C6, fact C7*)

8.3 Every request in the dialogues is answered

Fatio obliges the addressee of a question or a challenge to respond. With *Pending* this is checkable: both dialogues leave no request open, whereas their prefixes up to the response do.

theorem *BrigadeAnswered*: *Answered Brigade* **unfolding** *Answered-def* **by** *simp*
theorem *CascadeAnswered*: *Answered Cascade* **unfolding** *Answered-def* **by** *simp*
theorem *BrigadePrefixPending*: $\neg Answered$ (*take 2 Brigade*)
unfolding *Answered-def* **by** *simp*

8.4 Why the dialogue needs the generalised question pre-condition

Under the original, sign-blind pre-condition the fourth locution of the dialogue is not applicable: c addresses b , whose obligation stems from the challenge and is therefore negative, so no positive entry for b exists in the store. The dialogue is consequently rejected by the original checker while being accepted by the reconstructed one. This makes the deviation discussed in the protocol theory a theorem rather than a claim.

theorem *OriginalBlocksFourthMove*: $\neg \text{PreCondO } dos_3 \Gamma_3 \text{ question}[c, b, r^c]$
by *simp*

theorem *OriginalRejectsBrigade*: $\neg \text{FatioCheckRecO Brigade} [] \Gamma_B$
by *simp*

8.5 The examples are not vacuous

A consequence relation quantified over models is vacuously total on an unsatisfiable premise set: if no model validated Γ_B globally, every pre-condition would hold for trivial reasons and the dialogue checks above would say nothing. The following two theorems exclude this. The first exhibits a model of Γ_B (all accessibilities empty, so every belief and desire holds vacuously); the second shows that Γ_B nevertheless does not entail everything, using the same model with an empty valuation.

theorem Γ_B -satisfiable:
 $\forall \gamma. \Gamma_B \gamma \longrightarrow (\forall w: (\lambda w::w. \text{True}).$
 $\langle (\lambda w::w. \text{True}), (\lambda i w v. \text{False}), (\lambda i w v. \text{False}), V, U, E \rangle, w \models^d \gamma)$
by (*auto simp: Γ_B -def*)

theorem Γ_B -nonvacuous: $\neg (\Gamma_B \models q^a)$

proof

assume $A: \Gamma_B \models q^a$
let $?W = \lambda w::w. \text{True}$ **and** $?R = \lambda i w v. \text{False}$ **and** $?V = \lambda x w. \text{False}$
have *sat*: $\forall \gamma. \Gamma_B \gamma \longrightarrow (\forall w: ?W. \langle ?W, ?R, ?R, ?V, U, E \rangle, w \models^d \gamma)$
by (*auto simp: Γ_B -def*)
have *inst*: $(\forall \gamma. \Gamma_B \gamma \longrightarrow (\forall w: ?W. \langle ?W, ?R, ?R, ?V, U, E \rangle, w \models^d \gamma))$
 $\longrightarrow (\forall w: ?W. \langle ?W, ?R, ?R, ?V, U, E \rangle, w \models^d q^a)$
using $A[\text{unfolded ConsD-def},$
 $\text{THEN spec}[\text{where } x=?W], \text{ THEN spec}[\text{where } x=?R], \text{ THEN spec}[\text{where } x=?R],$
 $\text{ THEN spec}[\text{where } x=?V], \text{ THEN spec}[\text{where } x=U], \text{ THEN spec}[\text{where } x=E]]$ **by** *blast*
have $\forall w: ?W. \langle ?W, ?R, ?R, ?V, U, E \rangle, w \models^d q^a$ **using** *inst sat* **by** (*rule mp*)
thus *False* **by** *simp*
qed

8.6 Negative tests: the checker discriminates

A checker that accepted everything would also accept the dialogues above. These three theorems show that it does not: a justification without a preceding question or challenge, a retraction without a corresponding obligation, and an assertion repeated while the obligation is still open are all rejected.

theorem *NoJustifyWithoutRequest*: $\neg \text{PreCond } \text{dos}_1 \Gamma_1 \text{ justify}[a, n^c \oplus r^c]$
by (*auto simp*: $\Gamma_B\text{-def}$ *GammaUpdate-def*)

theorem *NoRetractWithoutObligation*: $\neg \text{PreCond } [] \Gamma_B \text{ retract}[a, r^c, \oplus]$
by *simp*

theorem *NoRepeatedAssert*: $\neg \text{PreCond } \text{dos}_1 \Gamma_1 \text{ assert}[a, r^c]$
by *simp*

theorem *BadDialogueRejected*: $\neg \text{FatioCheckRec } [\text{assert}[a, r^c], \text{assert}[a, r^c]] [] \Gamma_B$
by *simp*

8.7 A pre-condition that does not follow by membership

In the dialogues above every pre-condition is met because the world knowledge literally contains the required formula. Here the required desire is instead DERIVED, by the K principle transferred from the shallow embedding (see *ConsKDB* and *DeepKB*): the agent's knowledge contains an implication and its antecedent, not the conclusion.

definition $\Gamma_K :: \text{BDF} \Rightarrow \text{bool}$ **where** $\Gamma_K \equiv \lambda x.$
 $(\exists j. j \neq a \wedge x = \mathcal{D}^d a (\mathcal{B}^d j (\mathcal{B}^d a \{r^c\}^d \supset^d \mathcal{B}^d a \{s^c\}^d)))$
 $\vee (\exists j. j \neq a \wedge x = \mathcal{D}^d a (\mathcal{B}^d j (\mathcal{B}^d a \{r^c\}^d)))$

lemma *DerivedDesire*:

assumes *ja*: $j \neq a$

shows $\Gamma_K \models \mathcal{D}^d a (\mathcal{B}^d j (\mathcal{B}^d a \{s^c\}^d))$

proof –

have *imp*: $\Gamma_K \models \mathcal{D}^d a (\mathcal{B}^d j (\mathcal{B}^d a \{r^c\}^d \supset^d \mathcal{B}^d a \{s^c\}^d))$

using *ja* **by** (*intro MemberEntails*) (*auto simp*: $\Gamma_K\text{-def}$)

have *ant*: $\Gamma_K \models \mathcal{D}^d a (\mathcal{B}^d j (\mathcal{B}^d a \{r^c\}^d))$

using *ja* **by** (*intro MemberEntails*) (*auto simp*: $\Gamma_K\text{-def}$)

show *?thesis* **by** (*rule ConsKDB[OF imp ant]*)

qed

theorem *AssertByDerivation*: $\text{PreCond } [] \Gamma_K \text{ assert}[a, s^c]$
using *DerivedDesire* **by** *simp*

8.8 The same, established on the shallow side

The maximal shallow embedding put to work: the required desire is derived by reasoning with the shallow connectives only, and the result is transferred

to the deep consequence relation by *ConsD-via-shallow*. This is what the second backend is for; by faithfulness the two routes agree.

lemma *DerivedDesireShallow*:

```

  assumes ja:  $j \neq a$ 
  shows  $\Gamma_K \models \mathcal{D}^d a (\mathcal{B}^d j (\mathcal{B}^d a \{s^c\}^d))$ 
proof (rule ConsD-via-shallow)
  fix  $W::\mathcal{W}$  and  $B D::\mathcal{R}$  and  $V::\mathcal{V}$  and  $U::\mathcal{U}$  and  $E::\mathcal{E}$  and  $w::w$ 
  assume  $wW: W w$ 
  assume  $\text{satS}: \forall \gamma. \Gamma_K \gamma \longrightarrow (\forall v:W. \langle W, B, D, V, U, E \rangle, v \models^s \langle \gamma \rangle)$ 
  have  $m1: \Gamma_K (\mathcal{D}^d a (\mathcal{B}^d j (\mathcal{B}^d a \{r^c\}^d \supset^d \mathcal{B}^d a \{s^c\}^d)))$ 
    using ja by (auto simp:  $\Gamma_K\text{-def}$ )
  have  $m2: \Gamma_K (\mathcal{D}^d a (\mathcal{B}^d j (\mathcal{B}^d a \{r^c\}^d)))$ 
    using ja by (auto simp:  $\Gamma_K\text{-def}$ )
  have  $X: \langle W, B, D, V, U, E \rangle, w \models^s \langle \mathcal{D}^d a (\mathcal{B}^d j (\mathcal{B}^d a \{r^c\}^d \supset^d \mathcal{B}^d a \{s^c\}^d)) \rangle$ 
    using  $\text{satS}[\text{THEN spec, of } \mathcal{D}^d a (\mathcal{B}^d j (\mathcal{B}^d a \{r^c\}^d \supset^d \mathcal{B}^d a \{s^c\}^d))] m1 wW$ 
  by blast
  have  $Y: \langle W, B, D, V, U, E \rangle, w \models^s \langle \mathcal{D}^d a (\mathcal{B}^d j (\mathcal{B}^d a \{r^c\}^d)) \rangle$ 
    using  $\text{satS}[\text{THEN spec, of } \mathcal{D}^d a (\mathcal{B}^d j (\mathcal{B}^d a \{r^c\}^d))] m2 wW$  by blast
  show  $\langle W, B, D, V, U, E \rangle, w \models^s \langle \mathcal{D}^d a (\mathcal{B}^d j (\mathcal{B}^d a \{s^c\}^d)) \rangle$  using X Y by auto
qed

```

theorem *AssertByShallowDerivation*: $\text{PreCond} \sqcap \Gamma_K \text{ assert}[a, s^c]$

using *DerivedDesireShallow* by simp

theorem $\Gamma_C\text{-satisfiable}$:

```

 $\forall \gamma. \Gamma_C \gamma \longrightarrow (\forall w: (\lambda w::w. \text{True}).$ 
   $\langle (\lambda w. \text{True}), (\lambda x w v. \text{False}), (\lambda x w v. \text{False}), (\lambda x w. \text{False}), U_0, E_0 \rangle, w \models^d \gamma)$ 
  unfolding  $\Gamma_C\text{-def}$  by auto

```

end

References

- [1] C. Benzmüller. Faithful logic embeddings in HOL — deep and shallow (isabelle/hol dataset). *Archive of Formal Proofs*, May 2025. <https://isa-afp.org/entries/FaithfulPMLinHOL.html>, Formal proof development.
- [2] C. Benzmüller. Faithful logic embeddings in HOL – deep and shallow. In C. Barrett and U. Waldmann, editors, *Automated Deduction – CADE 30*, volume 15943 of *Lecture Notes in Computer Science*, pages 280–302. Springer, 2025. Preprint: arXiv:2502.19311.
- [3] C. Benzmüller, X. Parent, and L. van der Torre. Designing normative theories for ethical and legal reasoning: LogiKEy framework, methodology, and tool support. *Artificial Intelligence*, 287:103348, 2020.

- [4] J. C. Blanchette, S. Böhme, and L. C. Paulson. Extending sledgehammer with SMT solvers. In *Automated Deduction – CADE-23*, volume 6803 of *Lecture Notes in Computer Science*, pages 116–130. Springer, 2011.
- [5] J. C. Blanchette and T. Nipkow. Nitpick: A counterexample generator for higher-order logic based on a relational model finder. In M. Kaufmann and L. C. Paulson, editors, *Interactive Theorem Proving, ITP 2010*, volume 6172 of *Lecture Notes in Computer Science*, pages 131–146. Springer, 2010.
- [6] P. McBurney and S. Parsons. Locutions for argumentation in agent interaction protocols. In R. M. van Eijk, M.-P. Huget, and F. Dignum, editors, *Agent Communication, AC 2004*, volume 3396 of *Lecture Notes in Computer Science*, pages 209–225. Springer, 2005.
- [7] L. Pasetto and C. Benz Müller. Implementing the fatio protocol for multi-agent argumentation in LogiKEY. In C. Benz Müller, J. Otten, and R. Ramanayake, editors, *Proceedings of the 5th International Workshop on Automated Reasoning in Quantified Non-Classical Logics (ARQNL 2024) affiliated with the 12th International Joint Conference on Automated Reasoning (IJCAR 2024), Nancy, France, July 1, 2024*, volume 3875 of *CEUR Workshop Proceedings*, pages 38–47. CEUR-WS.org, 2024.